目录

目录 1
0.-FAQ 3
- The log file remains empty 3
- How to remove all debug statements at compile time ? 3
- How to use external fmt library instead of the bundled? 3
- Asynchronous logging hangs on application exit under Windows 3
- Asynchronous logging hangs under Windows dll 3
- How to use spdlog in shared library? 3
- Memory leak detected when using inside a shared library 3
- Colors do not appear when using custom format 3
- Source information do not appear when using custom format 3
- WinAPI min/max macro definitions 4
- Switch the default logger to stderr 4
- How to force log messages to be output (ignore log level filter) 4
- GCC standard library compatibility problem 4

1.-QuickStart 6
- Basic example 6
- Create an asynchronous logger using factory method 6
- Create an asynchronous logger and change thread pool settings 7
- Create multiple loggers that share the same file (sink) aka categories 7
- Create a logger with multiple sinks, each sink with its own formatting and log level 7
- Logging macros 8
- Log user-defined objects 8

1.1.-Thread-Safety 10
- Non thread safe functions 10
  - Loggers 10
  - Sinks 10

2.-Creating-loggers 11
- Creating loggers using the factory functions 11
- Accessing loggers using spdlog::get("...") 11
- Create a rotating file logger 11
- Creating asynchronous loggers 11
- Creating loggers manually 12
- Creating loggers with multiple sinks 12
- Creating multiple file loggers with the same output file 12

3.-Custom-formatting 13
- Customizing format using set_pattern(..) 13
- Efficiency 13
- Pattern flags 13
- Aligning 14
- Extending spdlog with your own flags 15
- Source location flags 15

4.-Sinks 17
- Available Sinks 17
- Implementing your own sink 30
- Adding sinks to the logger after creation: 31

5.-Logger-registry 32
- Registering new loggers 32
- Registry conflicts 32

Removing loggers from the registry 32
6.-Asynchronous-logging 33
Creating async loggers 33
Using the <spdlog::async_factory> template argument: 33
Using spdlog::create_async<Sink>: 33
Using spdlog::create_async_nb<Sink>: 33
Construct directly and use the global thread pool 33
Construct directly and use a custom thread pool 33
Note: the tp object in the above example must outlive the logger object, since the logger acquires a weak_ptr to the tp. 33
Full queue policy 33
spdlog's thread pool 33
Message ordering 34
Windows issues 34
7.-Flush-policy 35
Manual flush 35
Severity based flush 35
Interval based flush 35
8.-Tweaking 36
9.-CMake 38
CMake 38
Generate 38
Build/test 38
Install 38
Cross-compiling for Android with CMake 3.7 38
Use as static library with ExternalProject_Add as git submodule 39
Using spdlog in a shared library 39
Default-logger 41
Error-handling 42
Home 43
Welcome to spdlog wiki 43
How-to-use-spdlog-in-DLLs 44
Problem 44
Workaround 44
Integrating-into-Eclipse-IDE 46
Language-bindings 47
Language binding 47
Python 47
Installation from pypi.org 47
Source 47
Features 47
Makefile-Only 48
Makefile 48
Preventing-child-processes-from-inheriting-open-log-files 49
Preventing-undefined-thread-ids-in-child-processes 50
Setting-up-JSON-logging-with-spdlog 51
Windows-unicode-filenames 53

0.-FAQ

The log file remains empty

For performance reasons, log entries are not flushed to files immediately, but only after BUFSIZ bytes have been logged (as defined in libc).

Use any of the following methods to force flush:

```
my_logger->flush();                        // flush now
my_logger->flush_on(spdlog::level::info);      // auto flush when "info" or higher message is logged
spdlog::flush_on(spdlog::level::info);         // auto flush when "info" or higher message is logged on all loggers
spdlog::flush_every(std::chrono::seconds(5));  // flush periodically every 5 seconds (caution: must be _mt logger)
```

See the flush policy for details.

How to remove all debug statements at compile time ?

Define `SPDLOG_ACTIVE_LEVEL` to the desired log level before including spdlog.h and use the SPDLOG_ macros:

```
#define SPDLOG_ACTIVE_LEVEL SPDLOG_LEVEL_INFO // All DEBUG/TRACE statements will be removed by the pre-processor
#include "spdlog/spdlog.h"
...
SPDLOG_DEBUG("debug message to default logger"); // removed at compile time
SPDLOG_LOGGER_TRACE(my_logger, "trace message"); // removed at compile time
SPDLOG_INFO("info message to default logger");   // included
```

How to use external fmt library instead of the bundled?

To use external fmt library, `SPDLOG_FMT_EXTERNAL` macro must be defined.

If you build spdlog with CMake, you must define CMake variable `SPDLOG_FMT_EXTERNAL` or `SPDLOG_FMT_EXTERNAL_HO` (header-only fmt). `SPDLOG_FMT_EXTERNAL` macro is automatically defined in spdlog added by CMake.

Asynchronous logging hangs on application exit under Windows

At the end of main, call `spdlog::shutdown();`

Asynchronous logging hangs under Windows dll

Make sure you don't create the async logger (or the thread pool) inside DllMain (e.g. by using static logger variable inside the dll).

How to use spdlog in shared library?

See How to use spdlog in DLLs for details.

Memory leak detected when using inside a shared library

The async logger and `spdlog::flush_every()` create a worker thread.

When using these in a shared library, you must call `spdlog::shutdown();` before the shared library is unloaded from memory. Otherwise, the objects associated with the thread may not be released.

Colors do not appear when using custom format

You need enclose the colored part you want with `%^` and `%$`.

For example `spdlog::set_pattern("%^[%l]%$ %v");`

Source information do not appear when using custom format

Some custom format patterns, such as `%@`, `%s`, `%g`, `%#`, `%!`, print the source information.

For these to work, you must use compile time log level macros.

For example:

```
#define SPDLOG_ACTIVE_LEVEL SPDLOG_LEVEL_INFO
#include "spdlog/spdlog.h"
#include "spdlog/sinks/stdout_sinks.h"

void source_info_example()
{
   auto console = spdlog::stdout_logger_mt("console");
   spdlog::set_default_logger(console);
   spdlog::set_pattern("[source %s] [function %!] [line %#] %v");

   SPDLOG_LOGGER_INFO(console, "log with source info"); // Console: "[source example.cpp] [function source_info_example] [line 10] log with sou
   SPDLOG_INFO("global log with source info"); // Console: "[source example.cpp] [function source_info_example] [line 11] global logger with sour

   console->info("source info is not printed"); // Console: "[source ] [function ] [line ] source info is not printed"
}
```

WinAPI `min` / `max` macro definitions

When using the header-only distribution of spdlog together with the bundled fmt library on Windows you end up including `Windows.h` via the bundled fmt library. To prevent any errors caused by the definiton of `min` / `max` macros in WinAPI you can either

- Surround your `min()` or `max()` calls with parentheses: `(std::max)(n1, n2)` or
- define `NOMINMAX` macro before including spdlog.h, or
- use external fmt library(`-DSPDLOG_FMT_EXTERNAL=ON` when building spdlog)

See #1553 or fmt#1508 for details.

Switch the default logger to `stderr`

You can `set_default_logger()` with a new `stderr` logger. If you want the logging format to remain unchanged, you should specify the name as an empty string, but this will cause a conflict with the initial default logger, which is already using that name. To resolve this, first replace the default logger with one that has some other arbitrary name…

```
#include "spdlog/sinks/stdout_color_sinks.h"
#include "spdlog/spdlog.h"
int main()
{
   // Replace the default logger with a (color, single-threaded) stderr logger
   // (but first replace it with an arbitrarily-named logger to prevent a name clash)
   spdlog::set_default_logger( spdlog::stderr_color_st( "some_arbitrary_name" ) );
   spdlog::set_default_logger( spdlog::stderr_color_st( "" ) );
   spdlog::info("This message will go to stderr");
}
```

How to force log messages to be output (ignore log level filter)

You can use `spdlog::level::off` to force log messages to be output (Discussion).

```
logger.set_level(spdlog::level::error);

logger.warn("This message is not output.");

logger.log(spdlog::level::off, "This message is output.");
```

Note that this way does not trigger log level-based flush (`logger.flush_on(spdlog::level)`).

GCC standard library compatibility problem

If you build a simple code like the following with GCC compiler and get `std::bad_alloc`, you may be using an incompatible standard library.

```
#include "spdlog/spdlog.h"

int main(int argc, char *argv[])
{
   spdlog::info("Test message.");

   return 0;
}
```

`std::bad_alloc` occurs mostly when passing a dangling pointer or a string that has not been null terminated. However, in GCC only, this error may be reported when spdlog and the application use different standard libraries (e.g. `libstdc++` and `libstdc++11`). This problem has been reported more often when using the package manager. If you use a package manager, please check your ABI settings.

See #2286 or #2290 for details.

1.-QuickStart

```
#include "spdlog/spdlog.h"
int main()
{
   //Use the default logger (stdout, multi-threaded, colored)
   spdlog::info("Hello, {}!", "World");
}
```

spdlog is a header only library. Just copy the files under `include` to your build tree and use a C++11 compiler.

It provides a python like formatting API using the bundled `fmt` lib (see the reference):

```
logger->info("Hello {} {} !!", "param1", 123.4);
```

spdlog takes the "include what you need" approach - your code should include the features that are actually needed.

For example, if you only need rotating logger, you need to include " spdlog/sinks/rotating_file_sink.h".

Another example would be to include "spdlog/async.h" to get asynchronous logging features.

Basic example

```
#include <iostream>
#include "spdlog/spdlog.h"
#include "spdlog/sinks/basic_file_sink.h" // support for basic file logging
#include "spdlog/sinks/rotating_file_sink.h" // support for rotating file logging

int main(int, char* [])
{
   try
   {
      // Create basic file logger (not rotated)
      auto my_logger = spdlog::basic_logger_mt("basic_logger", "logs/basic.txt");

      // create a file rotating logger with 5mb size max and 3 rotated files
      auto file_logger = spdlog::rotating_logger_mt("file_logger", "myfilename", 1024 * 1024 * 5, 3);
   }
   catch (const spdlog::spdlog_ex& ex)
   {
      std::cout << "Log initialization failed: " << ex.what() << std::endl;
   }
}
```

Create an asynchronous logger using factory method

```
#include <iostream>
#include "spdlog/spdlog.h"
#include "spdlog/async.h" //support for async logging.
#include "spdlog/sinks/basic_file_sink.h"

int main(int, char* [])
{
   try
   {
      auto async_file = spdlog::basic_logger_mt<spdlog::async_factory>("async_file_logger", "logs/async_log.txt");
      for (int i = 1; i < 101; ++i)
      {
         async_file->info("Async message #{}", i);
      }
      // Under VisualStudio, this must be called before main finishes to workaround a known VS issue
      spdlog::drop_all();
   }
   catch (const spdlog::spdlog_ex& ex)
   {
```

```
        std::cout << "Log initialization failed: " << ex.what() << std::endl;
    }
}
```

Create an asynchronous logger and change thread pool settings

```
#include "spdlog/async.h" //support for async logging
#include "spdlog/sinks/daily_file_sink.h"
int main(int, char* [])
{
    try
    {
        auto daily_sink = std::make_shared<spdlog::sinks::daily_file_sink_mt>("logfile", 23, 59);
        // default thread pool settings can be modified *before* creating the async logger:
        spdlog::init_thread_pool(10000, 1); // queue with 10K items and 1 backing thread.
        auto async_file = spdlog::basic_logger_mt<spdlog::async_factory>("async_file_logger", "logs/async_log.txt");
        spdlog::drop_all();
    }
    catch (const spdlog::spdlog_ex& ex)
    {
        std::cout << "Log initialization failed: " << ex.what() << std::endl;
    }
}
```

Create multiple loggers that share the same file (sink) aka categories

```
#include <iostream>
#include "spdlog/spdlog.h"
#include "spdlog/sinks/daily_file_sink.h"
int main(int, char* [])
{
    try
    {
        auto daily_sink = std::make_shared<spdlog::sinks::daily_file_sink_mt>("logfile", 23, 59);
        // create synchronous  loggers
        auto net_logger = std::make_shared<spdlog::logger>("net", daily_sink);
        auto hw_logger  = std::make_shared<spdlog::logger>("hw",  daily_sink);
        auto db_logger  = std::make_shared<spdlog::logger>("db",  daily_sink);

        net_logger->set_level(spdlog::level::critical); // independent levels
        hw_logger->set_level(spdlog::level::debug);

        // globally register the loggers so they can be accessed using spdlog::get(logger_name)
        spdlog::register_logger(net_logger);
    }
    catch (const spdlog::spdlog_ex& ex)
    {
        std::cout << "Log initialization failed: " << ex.what() << std::endl;
    }
}
```

Create a logger with multiple sinks, each sink with its own formatting and log level

```
//
// Logger with console and file output.
// the console will show only warnings or worse, while the file will log all messages.
//
#include <iostream>
#include "spdlog/spdlog.h"
#include "spdlog/sinks/stdout_color_sinks.h" // or "../stdout_sinks.h" if no colors needed
#include "spdlog/sinks/basic_file_sink.h"
int main(int, char* [])
{
    try
    {
        auto console_sink = std::make_shared<spdlog::sinks::stdout_color_sink_mt>();
```

```
        console_sink->set_level(spdlog::level::warn);
        console_sink->set_pattern("[multi_sink_example] [%^%l%$] %v");

        auto file_sink = std::make_shared<spdlog::sinks::basic_file_sink_mt>("logs/multisink.txt", true);
        file_sink->set_level(spdlog::level::trace);

        spdlog::sinks_init_list sink_list = { file_sink, console_sink };

        spdlog::logger logger("multi_sink", sink_list.begin(), sink_list.end());
        logger.set_level(spdlog::level::debug);
        logger.warn("this should appear in both console and file");
        logger.info("this message should not appear in the console, only in the file");

        // or you can even set multi_sink logger as default logger
        spdlog::set_default_logger(std::make_shared<spdlog::logger>("multi_sink", spdlog::sinks_init_list({console_sink, file_sink})));

    }
    catch (const spdlog::spdlog_ex& ex)
    {
        std::cout << "Log initialization failed: " << ex.what() << std::endl;
    }
}
```

Logging macros

You can define SPDLOG_ACTIVE_LEVEL to the desired log level *before* including "spdlog.h".

This will turn on/off logging statements at compile time

```
#define SPDLOG_ACTIVE_LEVEL SPDLOG_LEVEL_DEBUG

spdlog::set_level(spdlog::level::debug); // or spdlog::set_level(spdlog::level::trace);

SPDLOG_LOGGER_TRACE(file_logger , "Some trace message that will not be evaluated.{} ,{}", 1, 3.23);
SPDLOG_LOGGER_DEBUG(file_logger , "Some Debug message that will be evaluated.. {} ,{}", 1, 3.23);
SPDLOG_DEBUG("Some debug message to default logger that will be evaluated");
```

Notice that `spdlog::set_level` is also necessary to print out debug or trace messages.

Log user-defined objects

```
#include "spdlog/spdlog.h"
#include "spdlog/fmt/ostr.h" // must be included
#include "spdlog/sinks/stdout_sinks.h"

class some_class {};
std::ostream& operator<<(std::ostream& os, const some_class& c)
{
    return os << "some_class";
}

void custom_class_example()
{
    some_class c;
    auto console = spdlog::stdout_logger_mt("console");
    console->info("custom class with operator<<: {}..", c);
}
```

or use the bundled `fmt` lib when advanced formatting is needed:

```
#include <iterator>

#include "spdlog/spdlog.h"
#include "spdlog/fmt/ostr.h" // must be included
#include "spdlog/sinks/stdout_sinks.h"

class some_class {
```

```
    int code;
};

template<typename OStream>
friend OStream &operator<<(OStream &os, const some_class& to_log)
{
    fmt::format_to(std::ostream_iterator<char>(os), "{:04X}", to_log.code);
    return os;
}

void custom_class_example()
{
    some_class c; c.code = 17;
    auto console = spdlog::stdout_logger_mt("console");
    console->info("custom class with operator<< using fmt: {}..", c);
}
```

1.1.-Thread-Safety

Non thread safe functions

The following functions should not be called concurrently from multiple threads on the same logger object:

- `set_error_handler(log_err_handler);`
- `logger::sinks()` - returns a reference to a non thread safe vector, so don't modify it concurrently (e.g. `logger->sinks().push_back(new_sink);`)

Note: This restriction applies to all kind of loggers ("_mt" or "_st").

Loggers

To create thread safe loggers, use the _mt factory functions.

For example:

```
auto logger = spdlog::basic_logger_mt(...);
```

To create single threaded loggers, use the _st factory functions.

For example:

```
auto logger = spdlog::basic_logger_st(...);
```

Sinks

- Thread safe sinks: sinks ending with `_mt` (e.g `daily_file_sink_mt`)
- Non thread safe sinks: sinks ending with `_st` (e.g `daily_file_sink_st`)

2.-Creating-loggers

Each logger contains a vector of one or more `std::shared_ptr<spdlog::sink>`.

On each log call (if the log level is right) the logger will call the `sink(log_msg)` function on each of them.

spdlog's sinks have `_mt` (multi threaded) or `_st` (single threaded) suffixes to indicate the thread safety. While single threaded sinks cannot be used from multiple threads simultaneously they might be faster because no locking is employed.

Creating loggers using the factory functions

```
//Create and return a shared_ptr to a multithreaded console logger.
#include "spdlog/sinks/stdout_color_sinks.h"
auto console = spdlog::stdout_color_mt("some_unique_name");
```

This will create a console logger, register it in spdlog's global registry with "some_unique_name" as its id, and return it as shared_ptr.

Accessing loggers using spdlog::get("...")

Loggers can be accessed from anywhere using the thread safe `spdlog::get("logger_name")` which returns a shared pointer.

Note: `spdlog::get` might slow your code down since it locks a mutex, so use with caution. It is advisable to save the `shared_ptr<spdlog::logger>` returned and use it directly, at least in hot code paths.

A good approach would be to have a private member of type `shared_ptr<spdlog::logger>` which will be set in the constructor:

```
class MyClass
{
private:
  std::shared_ptr<spdlog::logger> _logger;
public:
  MyClass()
  {
    //set _logger to some existing logger
    _logger = spdlog::get("some_logger");
    //or create directly
    //_logger = spdlog::rotating_file_logger_mt("my_logger", ...);
  }
};
```

Note 2: Manually created loggers (i.e. those you construct directly, see Creating loggers manually) do not get automatically registered and will not be found by the `get("...")` call.

If you want to register such logger use the `register_logger(...)` function:

```
spdlog::register_logger(my_logger);
...
auto the_same_logger = spdlog::get("mylogger");
```

Create a rotating file logger

```
//Create rotating file multi-threaded logger
#include "spdlog/sinks/rotating_file_sink.h"
auto file_logger = spdlog::rotating_logger_mt("file_logger", "logs/mylogfile", 1048576 * 5, 3);
...
auto same_logger= spdlog::get("file_logger");
```

Creating asynchronous loggers

```
#include "spdlog/async.h"
void async_example()
{
```

```
    // default thread pool settings can be modified *before* creating the async logger:
    // spdlog::init_thread_pool(8192, 1); // queue with 8k items and 1 backing thread.
    auto async_file = spdlog::basic_logger_mt<spdlog::async_factory>("async_file_logger", "logs/async_log.txt");
    // alternatively:
    // auto async_file = spdlog::create_async<spdlog::sinks::basic_file_sink_mt>("async_file_logger", "logs/async_log.txt");

}
```

For asynchronous logging, spdlog uses a shared global thread pool with a dedicated message queue.

For this, it creates fixed number of pre-allocated slots in the message queue (~256 bytes per slot in 64 bits) and can be modified using `spdlog::init_thread_pool(queue_size, backing_threads_count)`

When trying to log a message and the queue is full, the caller will block (default behavior) until a slot becomes available (default), or overrun oldest message immediately in the queue with the new one (if the logger was constructed with `async_overflow_policy==overrun_oldest`).

Creating loggers manually

```
auto sink = std::make_shared<spdlog::sinks::stdout_sink_mt>();
auto my_logger = std::make_shared<spdlog::logger>("mylogger", sink);

// Optionally register the logger.  This is only needed if you want to access it with spdlog::get("mylogger")
spdlog::register_logger(my_logger);
```

Creating loggers with multiple sinks

```
std::vector<spdlog::sink_ptr> sinks;
sinks.push_back(std::make_shared<spdlog::sinks::stdout_sink_st>());
sinks.push_back(std::make_shared<spdlog::sinks::daily_file_sink_st>("logfile", 23, 59));
auto combined_logger = std::make_shared<spdlog::logger>("name", begin(sinks), end(sinks));
//register it if you need to access it globally
spdlog::register_logger(combined_logger);
```

Creating multiple file loggers with the same output file

If you want to have different loggers to write to the same output file all of them must share the same sink. Otherwise you might get strange results.

```
auto sharedFileSink = std::make_shared<spdlog::sinks::basic_file_sink_mt>("fileName.txt");
auto firstLogger = std::make_shared<spdlog::logger>("firstLoggerName", sharedFileSink);
auto secondLogger = std::make_unique<spdlog::logger>("secondLoggerName", sharedFileSink);
```

3.-Custom-formatting

Each logger's sink has a formatter which formats the messages to its destination.

spdlog's default logging format is in the form of:

`[2014-10-31 23:46:59.678] [my_loggername] [info] Some message`

There are 2 ways to customize a logger's format:

- Set the pattern string (*recommended*):

```
set_pattern(pattern_string);
```

- Or implement custom formatter that implements the formatter interface and call

```
set_formatter(std::make_unique<my_custom_formatter>());
```

Customizing format using `set_pattern(..)`

Format can be applied globally to all registered loggers:

```
spdlog::set_pattern("*** [%H:%M:%S %z] [thread %t] %v ***");
```

or to a specific logger object:

```
some_logger->set_pattern(">>>>>>>>> %H:%M:%S %z %v <<<<<<<<<");
```

or to a specific sink object:

```
some_logger->sinks()[0]->set_pattern(">>>>>>>>> %H:%M:%S %z %v <<<<<<<<<");
some_logger->sinks()[1]->set_pattern("..");
```

Efficiency

Whenever the user calls `set_pattern("..")` the library "compiles" the new pattern to an internal efficient representation - This way the performance stays excellent even with complex patterns (no re-parsing of the pattern on each log call).

Pattern flags

Pattern flags are in the form of `%flag` and resembles the strftime function:

flag	meaning	example
`%v`	The actual text to log	"some user text"
`%t`	Thread id	"1232"
`%P`	Process id	"3456"
`%n`	Logger's name	"some logger name"
`%l`	The log level of the message	"debug", "info", etc
`%L`	Short log level of the message	"D", "I", etc
`%a`	Abbreviated weekday name	"Thu"
`%A`	Full weekday name	"Thursday"
`%b`	Abbreviated month name	"Aug"
`%B`	Full month name	"August"

`%c`	Date and time representation	"Thu Aug 23 15:35:46 2014"
`%C`	Year in 2 digits	"14"
`%Y`	Year in 4 digits	"2014"
`%D` or `%x`	Short MM/DD/YY date	"08/23/14"
`%m`	Month 01-12	"11"
`%d`	Day of month 01-31	"29"
`%H`	Hours in 24 format 00-23	"23"
`%I`	Hours in 12 format 01-12	"11"
`%M`	Minutes 00-59	"59"
`%S`	Seconds 00-59	"58"
`%e`	Millisecond part of the current second 000-999	"678"
`%f`	Microsecond part of the current second 000000-999999	"056789"
`%F`	Nanosecond part of the current second 000000000-999999999	"256789123"
`%p`	AM/PM	"AM"
`%r`	12 hour clock	"02:55:02 PM"
`%R`	24-hour HH:MM time, equivalent to %H:%M	"23:55"
`%T` or `%X`	ISO 8601 time format (HH:MM:SS), equivalent to %H:%M:%S	"23:55:59"
`%z`	ISO 8601 offset from UTC in timezone ([+/-]HH:MM)	"+02:00"
`%E`	Seconds since the epoch	"1528834770"
`%%`	The % sign	"%"
`%+`	spdlog's default format	"[2014-10-31 23:46:59.678] [mylogger] [info] Some message"
`%^`	start color range (can be used only once)	"[mylogger] [info(green)] Some message"
`%$`	end color range (for example %^[+++]%$ %v) (can be used only once)	[+++] Some message
`%@`	Source file and line (use SPDLOG_TRACE(..), SPDLOG_INFO(...) etc. instead of spdlog::trace(...)) Same as %g:%#	/some/dir/my_file.cpp:123
`%s`	Basename of the source file (use SPDLOG_TRACE(..), SPDLOG_INFO(...) etc.)	my_file.cpp
`%g`	Full or relative path of the source file as appears in the `__FILE__` macro (use SPDLOG_TRACE(..), SPDLOG_INFO(...) etc.)	/some/dir/my_file.cpp
`%#`	Source line (use SPDLOG_TRACE(..), SPDLOG_INFO(...) etc.)	123
`%!`	Source function (use SPDLOG_TRACE(..), SPDLOG_INFO(...) etc. see tweakme for pretty-print)	my_func
`%o`	Elapsed time in milliseconds since previous message	456
`%i`	Elapsed time in microseconds since previous message	456
`%u`	Elapsed time in nanoseconds since previous message	11456
`%O`	Elapsed time in seconds since previous message	4

Aligning

Each pattern flag can be aligned by prepending a width number(up to 64).

Use `-` (left align) or `=` (center align) to control the align side:

align	meaning	example	result
`%<width><flag>`	Right align	`%8l`	" info"
`%-<width><flag>`	Left align	`%-8l`	"info "
`%=<width><flag>`	Center align	`%=8l`	" info "

Optionally add `!` to truncate the result if its size exceeds the specified width:

align	meaning	example	result
`%<width>!<flag>`	Right align or truncate	`%3!l`	"inf"
`%-<width>!<flag>`	Left align or truncate	`%-2!l`	"in"
`%=<width>!<flag>`	Center align or truncate	`%=1!l`	"i"

Note: To truncate function names use '!!'. For example `%10!!` will limit function names to 10 chars.

Extending spdlog with your own flags

You can define your own flags by inheriting the custom_flag_formatter class and implementing the `clone()` and `format(...)` abstract methods.

The following example adds a new flag %* - which will be bound to a <my_formatter_flag> instance.

```
#include "spdlog/pattern_formatter.h"
class my_formatter_flag : public spdlog::custom_flag_formatter
{
public:
    void format(const spdlog::details::log_msg &, const std::tm &, spdlog::memory_buf_t &dest) override
    {
        std::string some_txt = "custom-flag";
        dest.append(some_txt.data(), some_txt.data() + some_txt.size());
    }

    std::unique_ptr<custom_flag_formatter> clone() const override
    {
        return spdlog::details::make_unique<my_formatter_flag>();
    }
};

void custom_flags_example()
{
    auto formatter = std::make_unique<spdlog::pattern_formatter>();
    formatter->add_flag<my_formatter_flag>('*').set_pattern("[%n] [%*] [%^%l%$] %v");
    spdlog::set_formatter(std::move(formatter));
}
```

Note: The `clone()` method should return a deep copy of the object and is required because spdlog passes a new copy of the object to each sink in use (for performance reasons as this enables to have a state in this object without worrying about race conditions or thread safety across sinks).

Note: You can also override spdlog's built in flags in this way.

Source location flags

If you need to use source location flags like `%s`, `%g`, `%#`, `%!`, is necessary to define this compiler flag:

```
#define SPDLOG_ACTIVE_LEVEL SPDLOG_LEVEL_TRACE
```

(change the log level according to your needs) and use these macros:

```
SPDLOG_LOGGER_TRACE(some_logger, "trace message");
SPDLOG_LOGGER_DEBUG(some_logger, "debug message");
SPDLOG_LOGGER_INFO(some_logger, "info message");
SPDLOG_LOGGER_WARN(some_logger, "warn message");
```

```
SPDLOG_LOGGER_ERROR(some_logger, "error message");
SPDLOG_LOGGER_CRITICAL(some_logger, "critical message");
```

NOTE: if you've set the flag to `SPDLOG_LEVEL_TRACE` but you don't see any trace and/or debug messages, it's most likely something like this https://github.com/gabime/spdlog/issues/2764. In that case define it directly as a compiler flag. For example, using cmake:

```
add_compile_definitions(SPDLOG_ACTIVE_LEVEL=SPDLOG_LEVEL_TRACE)
```

4.-Sinks

Sinks are the objects that actually write the log to their target. Each sink should be responsible for only single target (e.g file, console, db), and each sink has its own private instance of formatter object.

Each logger contains a vector of one or more `std::shared_ptr<sink>`. On each log call (if the log level is right) the logger will call the `"sink(log_msg)"` function on each of them.

spdlog's sinks have _mt (multi threaded) or _st (single threaded) suffixes to indicate the thread safety. While single threaded sinks cannot be used from multiple threads simultaneously, they are faster because no locking is employed.

Available Sinks

Note This is only a partial list. For the full list of sinks please visit the sinks folder.

Note: Starting from version 1.5.0, spdlog will create, if needed, the folders that contain the log file. Prior to this, the folders must be created manually.

rotating_file_sink

When the max file size is reached, close the file, rename it, and create a new file. Both the max file size and the max number of files are configurable in the constructor.

```
// create a thread safe sink which will keep its file size to a maximum of 5MB and a maximum of 3 rotated files.
#include "spdlog/sinks/rotating_file_sink.h"
...
auto file_logger = spdlog::rotating_logger_mt("file_logger", "logs/mylogfile", 1048576 * 5, 3);
```

or create the sink manually and pass it to the logger:

```
#include "spdlog/sinks/rotating_file_sink.h"
...
auto rotating = make_shared<spdlog::sinks::rotating_file_sink_mt> ("log_filename", 1024*1024, 5, false);
auto file_logger = make_shared<spdlog::logger>("my_logger", rotating);
```

daily_file_sink

Create a new log file every day at the specified time, appending a timestamp to the filename.

```
#include "spdlog/sinks/daily_file_sink.h"
..
auto daily_logger = spdlog::daily_logger_mt("daily_logger", "logs/daily", 14, 55);
```

will create a thread safe sink which will create a new log file each day on 14:55.

Note: Starting from version 1.5.0, spdlog will create, if needed, the folders that contain the log file. Prior to this, the folders must be created manually.

simple_file_sink

A simple file sink that just writes to the given log file with no restrictions.

```
#include "spdlog/sinks/basic_file_sink.h"
...
auto logger = spdlog::basic_logger_mt("mylogger", "log.txt");
```

Note: Starting from version 1.5.0, spdlog will create, if needed, the folders that contain the log file. Prior to this, the folders must be created manually.

stdout_sink/stderr_sink

```
#include "spdlog/sinks/stdout_sinks.h"
...
auto console = spdlog::stdout_logger_mt("console");
auto err_console = spdlog::stderr_logger_st("console");
```

stdout_sink/stderr_sink with colors

```
#include "spdlog/sinks/stdout_color_sinks.h"
...
auto console = spdlog::stdout_color_mt("console");
auto err_console = spdlog::stderr_color_st("console");
```

Or create the sinks directly:

```
auto sink = std::make_shared<spdlog::sinks::stdout_color_sink_mt>();
```

ostream_sink

```
#include "spdlog/sinks/ostream_sink.h "
...
std::ostringstream oss;
auto ostream_sink = std::make_shared<spdlog::sinks::ostream_sink_mt> (oss);
auto logger = std::make_shared<spdlog::logger>("my_logger", ostream_sink);
```

null_sink:

null sink that throws away its log - can be used for debugging or as a reference implementation.

```
#include "spdlog/sinks/null_sink.h"
...
auto logger = spdlog::create<spdlog::sinks::null_sink_st>("null_logger");
```

syslog_sink

POSIX syslog(3) sink which sends its log to syslog.

```
#include "spdlog/sinks/syslog_sink.h"
...
auto syslog_sink = std::make_shared<spdlog::sinks::syslog_sink_mt>("my_ident", LOG_PID);
auto syslog_logger = std::make_shared<spdlog::logger>("logger_name", syslog_sink);
```

systemd_sink

sink which sends its log to systemd

```
#include "spdlog/sinks/systemd_sink.h"
...
auto systemd_sink = std::make_shared<spdlog::sinks::systemd_sink_st>();
auto systemd_logger = std::make_shared<spdlog::logger>("logger_name", systemd_sink);
```

dist_sink (sinks/dist_sink.h):

Distribute log messages to a list of other sinks:

```
#include "spdlog/sinks/dist_sink.h"

...
auto dist_sink = make_shared<spdlog::sinks::dist_sink_st>();
auto sink1 = make_shared<spdlog::sinks::stdout_sink_st>();
auto sink2 = make_shared<spdlog::sinks::simple_file_sink_st>("mylog.log");

dist_sink->add_sink(sink1);
dist_sink->add_sink(sink2);
```

msvc_sink Windows debug sink (logging using OutputDebugStringA):

```
#include "spdlog/sinks/msvc_sink.h"
auto sink = std::make_shared<spdlog::sinks::msvc_sink_mt>();
auto logger = std::make_shared<spdlog::logger>("msvc_logger", sink);
```

dup_filter_sink Duplicate messages removal sink. Skip the message if previous one is identical and less than "max_skip_duration" have passed.

Example:

```
#include "spdlog/sinks/dup_filter_sink.h"

auto dup_filter = std::make_shared<dup_filter_sink_st>(std::chrono::seconds(5));
dup_filter->add_sink(std::make_shared<stdout_color_sink_mt>());
spdlog::logger l("logger", dup_filter);
l.info("Hello");
l.info("Hello");
l.info("Hello");
l.info("Different Hello");
```

Will produce:

```
[2019-06-25 17:50:56.511] [logger] [info] Hello
[2019-06-25 17:50:56.512] [logger] [info] Skipped 3 duplicate messages..
[2019-06-25 17:50:56.512] [logger] [info] Different Hello
```

ringbuffer_sink

ringbuffer sink keeps most recent log messages in memory. To retrieve a log message, call `spdlog::sinks::ringbuffer_sink::last_formatted(size_t)`.

Example:

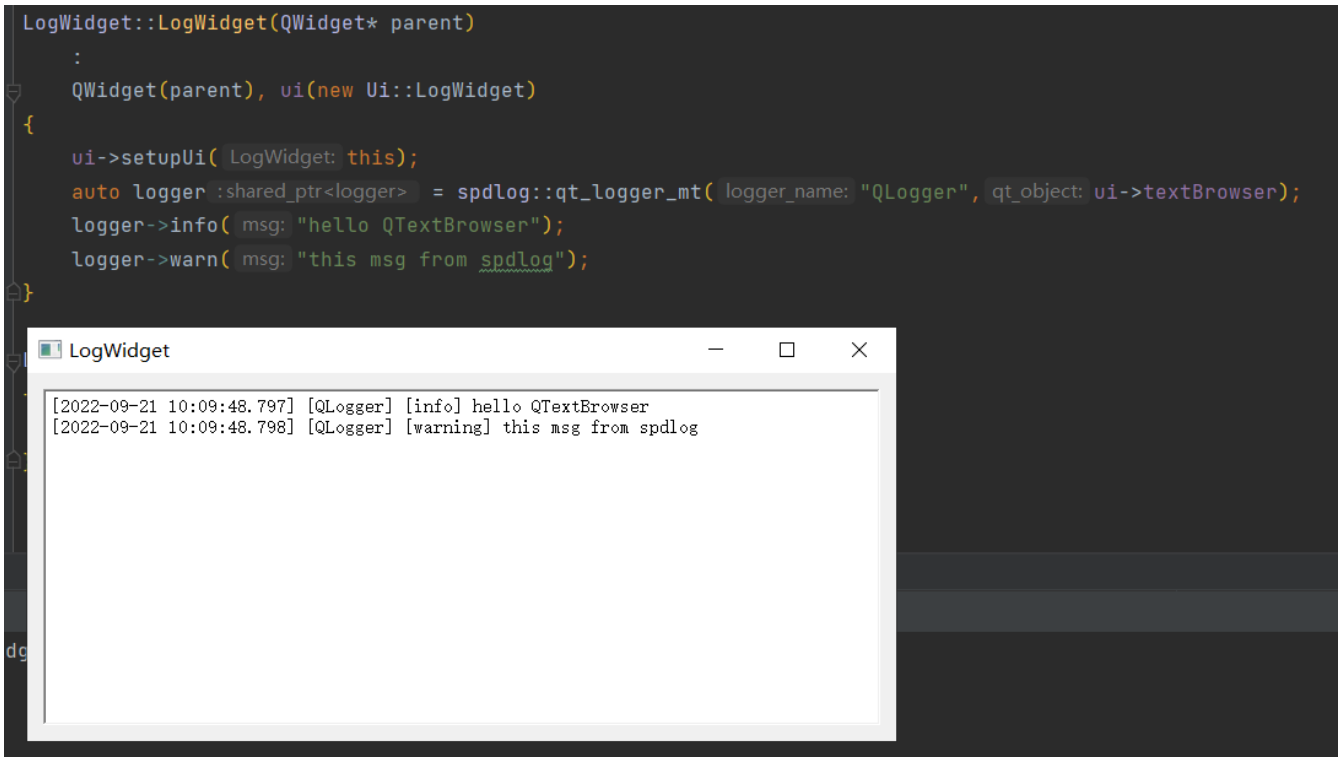
```
#include "spdlog/sinks/ringbuffer_sink.h"

auto ringbuffer_sink = std::make_shared<spdlog::sinks::ringbuffer_sink_mt>(128);

std::vector<spdlog::sink_ptr> sinks;
sinks.push_back(std::make_shared<spdlog::sinks::basic_file_sink_mt>("path/to/log.txt"));
sinks.push_back(ringbuffer_sink);

auto logger = std::make_shared<spdlog::logger>("logger_name", std::begin(sinks), std::end(sinks));

for (int i = 0; i < 256; ++i) {
    logger->info("Log message {}", i);
}

// Retrieve all log messages. `log_message` contains 128 messages.
std::vector<std::string> log_messages = ringbuffer_sink->last_formatted();
// Retrieve a maximum of 64 log messages.
std::vector<std::string> log_messages = ringbuffer_sink->last_formatted(64);
```

qt_sink

qt_sink could output log messages from QTextBrowser,QTextEdit...

Example:

```
#include "spdlog/sinks/qt_sinks.h"
auto logger = spdlog::qt_logger_mt("QLogger",ui->textBrowser);
logger->info("hello QTextBrowser");
logger->warn("this msg from spdlog");
```



Implementing your own sink

To implement your own sink, you'll need to implement the simple sink interface.

A recommended way is to inherit from the base_sink class. This class already handles thread locking, and makes it very easy to implement a thread safe sink.

In this case you only need to implement the protected "sink_it_(..)" and flush_(..) functions:

```
#include "spdlog/sinks/base_sink.h"

template<typename Mutex>
class my_sink : public spdlog::sinks::base_sink <Mutex>
{
...
protected:
   void sink_it_(const spdlog::details::log_msg& msg) override
   {

   // log_msg is a struct containing the log entry info like level, timestamp, thread id etc.
   // msg.raw contains pre formatted log

   // If needed (very likely but not mandatory), the sink formats the message before sending it to its final destination:
   spdlog::memory_buf_t formatted;
   spdlog::sinks::base_sink<Mutex>::formatter_->format(msg, formatted);
   std::cout << fmt::to_string(formatted);
   }
```

```
    void flush_() override
    {
      std::cout << std::flush;
    }
};

#include "spdlog/details/null_mutex.h"
#include <mutex>
using my_sink_mt = my_sink<std::mutex>;
using my_sink_st = my_sink<spdlog::details::null_mutex>;
```

Adding sinks to the logger after creation:

As of spdlog v1.x there is a function returning a non-const reference to the sinks vector, allowing you to manually push back sinks. There is no mutex protecting the sinks vector (which is obvious due to the performance impact) so it's NOT thread-safe! Also see: https://github.com/gabime/spdlog/wiki/1.1.-Thread-Safety

```
inline std::vector<spdlog::sink_ptr> &spdlog::logger::sinks()
{
    return sinks_;
}
```

```
spdlog::get("myExistingLogger")->sinks().push_back(myNewSink);
```

5.-Logger-registry

spdlog maintains a global (per process) registry of the created loggers.

The purpose is for loggers to be accessed easily from anywhere in the project without passing them around.

```
spdlog::get("logger1")->info("hello");
..
..
some other source file..
..
auto l = spdlog::get("logger1");
l->info("hello again");
```

If a logger is not found, an empty shared pointer is returned. You use `if(l)` to check the validity of the pointer `l`.

Registering new loggers

Normally there is no need to register loggers as they are registered automatically for you.

To register manually created loggers(i.e. not created by the spdlog.h factory functions) use the `register_logger(std::shared_ptr<logger>)` function:

```
spdlog::register_logger(some_logger);
```

which will register `some_logger` using its name.

Registry conflicts

spdlog will throw a `spdlog::spdlog_ex` exception on attempting to register using a name that already exists in the registry

Removing loggers from the registry

The "drop()" function can be used to remove a logger from the registry.

If no other shared_ptr pointing to the logger exists, the logger will be closed and all its resources will be freed.

```
spdlog::drop("logger_name");
//or remove them all
spdlog::drop_all()
```

6.-Asynchronous-logging

Creating async loggers

There are several ways to create an asynchronous logger. In all ways you must `#include spdlog/async.h`:

Using the `<spdlog::async_factory>` template argument:

```
#include "spdlog/async.h"
void async_example()
{
   // default thread pool settings can be modified *before* creating the async logger:
   // spdlog::init_thread_pool(8192, 1); // queue with 8k items and 1 backing thread.
   auto async_file = spdlog::basic_logger_mt<spdlog::async_factory>("async_file_logger", "logs/async_log.txt");
}
```

Using `spdlog::create_async<Sink>`:

```
auto async_file = spdlog::create_async<spdlog::sinks::basic_file_sink_mt>("async_file_logger", "logs/async_log.txt");
```

Using `spdlog::create_async_nb<Sink>`:

This will create a logger that will never block on full queue.

```
auto async_file = spdlog::create_async_nb<spdlog::sinks::basic_file_sink_mt>("async_file_logger", "logs/async_log.txt");
```

Construct directly and use the global thread pool

```
spdlog::init_thread_pool(queue_size, n_threads);
auto logger = std::make_shared<spdlog::async_logger>("as", some_sink, spdlog::thread_pool(), async_overflow_policy::block);
```

Construct directly and use a custom thread pool

```
auto tp = std::make_shared<details::thread_pool>(queue_size, n_threads);
auto logger = std::make_shared<spdlog::async_logger>("as", some_sink, tp, async_overflow_policy::block);
```

Note: the tp object in the above example must outlive the logger object, since the logger acquires a weak_ptr to the tp.

Full queue policy

There are 2 options on what to do when the queue is full:

- Block the caller until there is more room (default behaviour)
- Drop and replace the oldest messages in the queue with new ones instead of waiting for more room. Use the `create_async_nb` factory function or the `spdlog::async_overflow_policy` in the logger's constructor:

```
auto logger = spdlog::create_async_nb<spdlog::sinks::basic_file_sink_mt>("async_file_logger", "logs/async_log.txt");
// or directly:
 auto logger = std::make_shared<async_logger>("as", test_sink, spdlog::thread_pool(), spdlog::async_overflow_policy::overrun_oldest);
```

spdlog's thread pool

By default, spdlog create a global thread pool with queue size of 8192 and 1 worker thread which to serve all async loggers.

This means that creating and destructing async loggers is cheap, since they do not own nor create any backing threads or queues- these are created and managed by the shared thread pool object.

All the queue slots are pre-allocated on thread-pool construction (each slot occupies ~256 bytes on 64bit systems)

The thread pool size and threads can be reset by

```
spdlog::init_thread_pool(queue_size, n_threads);
```

Note that this will drop the old global thread pool (tp) and create a new tp - which means that any loggers that use the old tp will stop working, so it is advised calling it before any async loggers created.

If different loggers must have separate queues, different instances of pools can be created and passed them to the loggers:

```
auto tp = std::make_shared<details::thread_pool>(128, 1);
auto logger = std::make_shared<async_logger>("as", some_sink, tp, async_overflow_policy::overrun_oldest);

auto tp2 = std::make_shared<details::thread_pool>(1024, 4);  // create pool with queue of 1024 slots and 4 backing threads
auto logger2 = std::make_shared<async_logger>("as2", some_sink, tp2, async_overflow_policy::block);
```

Thread-pool has additional constructors that take callbacks as arguments. Those will be executed by each thread following its creation and prior to its destruction.

```
std::function<void()> on_start = []() { /* execute on start */ };
auto on_stop = []() { /* execute on stop */ };
auto tp = std::make_shared<details::thread_pool>(1024, 1, on_start, on_stop);

// init_thread_pool helper also supports optional on_start and on_stop callbacks
spdlog::init_thread_pool(1024, 1, on_start);
spdlog::init_thread_pool(1024, 1, on_start, on_stop);
```

Message ordering

By default, spdlog creates a single worker thread which preserves the order of the messages in the queue.

Please note that with thread pool with multiple workers threads, messages may be reordered after dequeuing.

If you want to keep the message order, create only one worker thread in the thread pool.

Windows issues

There is a bug in VS runtime that cause the application dead lock when it exits. If you use async logging, please make sure to call `spdlog::shutdown()` before main() exit (stackoverflow: std::thread join hangs if called after main exits when using vs2012 rc).

7.-Flush-policy

By default spdlog lets the underlying libc flush whenever it sees fit in order to achieve good performance. You can override this using the following options:

Manual flush

You can use the `logger->flush()` function to instruct a logger to flush its contents. The logger will in turn call the `flush()` function on each of the underlying sinks.

Note: If using an async logger, `logger->flush()` posts a message to the queue requesting the flush operation, so the function returns immediately. This is different from some older versions of spdlog (which would synchronously wait until the message was received and the flush completed). Currently there is no need to explicitly call `logger->flush()` or `spdlog::shutdown()` before shutting down, it is done automatically at destruction while program exits. However, if you want to flush all the async logger manually before "immediately" exit function such as `abort()` or `_exit(-1)`, please call `spdlog::shutdown()` before those functions.

Severity based flush

You can set the minimum log level that will trigger automatic flush.

For example, this will trigger flush whenever errors or more severe messages are logged:

```
my_logger->flush_on(spdlog::level::err);
```

Interval based flush

spdlog supports setting flush interval. This is implemented by a single worker thread that periodically calls flush() on each logger.

For example, turn on periodic flush with interval of 5 seconds for all registered loggers:

```
spdlog::flush_every(std::chrono::seconds(5));
```

Note Use this only on thread safe loggers, since the periodic flush happens from a different thread.

8.-Tweaking

To squeeze every last drop of performance out of spdlog, the tweakme.h header file can be edited.

Here is an example which let spdlog use Linux's CLOCK_REALTIME_COARSE fast clock

```
///////////////////////////////////////////////////////////////////////////////
//
// Edit this file to squeeze more performance, and to customize supported
// features
//
///////////////////////////////////////////////////////////////////////////////

///////////////////////////////////////////////////////////////////////////////
// Under Linux, the much faster CLOCK_REALTIME_COARSE clock can be used.
// This clock is less accurate - can be off by dozens of millis - depending on
// the kernel HZ.
// Uncomment to use it instead of the regular clock.
//
#define SPDLOG_CLOCK_COARSE
///////////////////////////////////////////////////////////////////////////////

///////////////////////////////////////////////////////////////////////////////
// Uncomment if thread id logging is not needed (i.e. no %t in the log pattern).
// This will prevent spdlog from querying the thread id on each log call.
//
// WARNING: If the log pattern contains thread id (i.e, %t) while this flag is
// on, the result is undefined.
//
// #define SPDLOG_NO_THREAD_ID
///////////////////////////////////////////////////////////////////////////////

///////////////////////////////////////////////////////////////////////////////
// Uncomment to prevent spdlog from using thread local storage.
//
// WARNING: if your program forks, UNCOMMENT this flag to prevent undefined
// thread ids in the children logs.
//
// #define SPDLOG_NO_TLS
///////////////////////////////////////////////////////////////////////////////

///////////////////////////////////////////////////////////////////////////////
// Uncomment to avoid spdlog's usage of atomic log levels
// Use only if your code never modifies a logger's log levels concurrently by
// different threads.
//
// #define SPDLOG_NO_ATOMIC_LEVELS
///////////////////////////////////////////////////////////////////////////////

///////////////////////////////////////////////////////////////////////////////
// Uncomment to enable usage of wchar_t for file names on Windows.
//
// #define SPDLOG_WCHAR_FILENAMES
///////////////////////////////////////////////////////////////////////////////

///////////////////////////////////////////////////////////////////////////////
// Uncomment to override default eol ("\n" or "\r\n" under Linux/Windows)
//
// #define SPDLOG_EOL ";-)\n"
///////////////////////////////////////////////////////////////////////////////

///////////////////////////////////////////////////////////////////////////////
// Uncomment to use your own copy of the fmt library instead of spdlog's copy.
// In this case spdlog will try to include <fmt/format.h> so set your -I flag
// accordingly.
//
// #define SPDLOG_FMT_EXTERNAL
///////////////////////////////////////////////////////////////////////////////
```

```
///////////////////////////////////////////////////////////////////////////////
// Uncomment to enable wchar_t support (convert to utf8)
//
// #define SPDLOG_WCHAR_TO_UTF8_SUPPORT
///////////////////////////////////////////////////////////////////////////////

///////////////////////////////////////////////////////////////////////////////
// Uncomment to prevent child processes from inheriting log file descriptors
//
// #define SPDLOG_PREVENT_CHILD_FD
///////////////////////////////////////////////////////////////////////////////

///////////////////////////////////////////////////////////////////////////////
// Uncomment to customize level names (e.g. "MT TRACE")
//
// #define SPDLOG_LEVEL_NAMES { "MY TRACE", "MY DEBUG", "MY INFO", "MY WARNING",
// "MY ERROR", "MY CRITICAL", "OFF" }
///////////////////////////////////////////////////////////////////////////////

///////////////////////////////////////////////////////////////////////////////
// Uncomment to customize short level names (e.g. "MT")
// These can be longer than one character.
//
// #define SPDLOG_SHORT_LEVEL_NAMES { "T", "D", "I", "W", "E", "C", "O" }
///////////////////////////////////////////////////////////////////////////////

///////////////////////////////////////////////////////////////////////////////
// Uncomment to disable default logger creation.
// This might save some (very) small initialization time if no default logger is needed.
//
// #define SPDLOG_DISABLE_DEFAULT_LOGGER
///////////////////////////////////////////////////////////////////////////////

///////////////////////////////////////////////////////////////////////////////
// Uncomment and set to compile time level with zero cost (default is INFO).
// Macros like SPDLOG_DEBUG(..), SPDLOG_INFO(..)  will expand to empty statements if not enabled
//
// #define SPDLOG_ACTIVE_LEVEL SPDLOG_LEVEL_INFO
///////////////////////////////////////////////////////////////////////////////

///////////////////////////////////////////////////////////////////////////////
// Uncomment (and change if desired) macro to use for function names.
// This is compiler dependent.
// __PRETTY_FUNCTION__ might be nicer in clang/gcc, and __FUNCTION__ in msvc.
// Defaults to __FUNCTION__ (should work on all compilers) if not defined.
//
// #define SPDLOG_FUNCTION __PRETTY_FUNCTION__
///////////////////////////////////////////////////////////////////////////////
```

9.-CMake

CMake

The library can also be installed using CMake. Version of CMake must be 3.0+ and can be checked by `--version`:

```
> cmake --version
cmake version 3.3.1

CMake suite maintained and supported by Kitware (kitware.com/cmake).
```

Generate

Generate project in `_builds` directory using `CMakeLists.txt` from current (`.`) directory, run one of the commands depending on the OS and generator you need:

```
> cmake -H. -B_builds -DCMAKE_BUILD_TYPE=Release # Default generator is Makefile for *nix platform
> cmake -H. -B_builds -GXcode # Xcode generator on OS X (multi-config, no need for CMAKE_BUILD_TYPE)
> cmake -H. -B_builds -G"Visual Studio 12 2013" # Visual Studio IDE, Windows, multi-config
```

or toolchains:

```
> cmake -H. -B_builds -DCMAKE_TOOLCHAIN_FILE=/path/to/iOS.cmake -GXcode
> cmake -H. -B_builds -DCMAKE_TOOLCHAIN_FILE=/path/to/android.cmake -DCMAKE_BUILD_TYPE=Release
```

To add examples use `SPDLOG_BUILD_EXAMPLES=ON` option:

```
> cmake -H. -B_builds -DSPDLOG_BUILD_EXAMPLES=ON -DCMAKE_BUILD_TYPE=Release
```

Build/test

Open generated project and run build/test targets. Also command line can be used too:

```
> cmake --build _builds --config Release
> cd _builds && ctest -VV -C Release
```

Install

Headers can be installed along with the config files so library can be found by `find_package(spdlog CONFIG REQUIRED)`:

```
[spdlog]> cmake -H. -B_builds -DCMAKE_INSTALL_PREFIX=/path/to/install -DCMAKE_BUILD_TYPE=Release
[spdlog]> cmake --build _builds --target install
```

Usage can be tested by building `examples` as a stand-alone project:

```
[spdlog/examples]> cmake -H. -B_builds -DCMAKE_PREFIX_PATH=/path/to/install -DCMAKE_BUILD_TYPE=Release
[spdlog/examples]> cmake --build _builds
```

Cross-compiling for Android with CMake 3.7

Cross-compiling with CMake is reported working using CMake version 3.7 which includes native support for configuring the toolchain with Android NDK and SDK (https://cmake.org/cmake/help/v3.7/manual/cmake-toolchains.7.html#cross-compiling-for-android). Please note that spdlog targets older versions of CMake and this example is a community contribution.

Below is an example on to configure and build spdlog using the Android NDK:

```
> git clone https://github.com/gabime/spdlog
> mkdir build
```

```
> cd build
> cmake ../spdlog \
  -DCMAKE_SYSTEM_NAME=Android \
  -DCMAKE_SYSTEM_VERSION=21 \
  -DCMAKE_ANDROID_ARCH_ABI="armeabi-v7a" \
  -DCMAKE_ANDROID_ARM_NEON=ON \
  -DCMAKE_ANDROID_NDK=$HOME/Android/Ndk/android-ndk-r13b/
> cmake --build .
```

See the CMake documentation (link above) for the meaning of the CMake configuration options used in the example.

Use as static library with ExternalProject_Add as git submodule

This is a short example if you want to use a static library, actually called libspdlogd.a, in your project using CMake.

```
cd project_dir\lib
git submodule add https://github.com/gabime/spdlog
cd spdlog
git checkout v1.9.2
...
```

```
ExternalProject_Add(spdlog
    PREFIX spdlog
    SOURCE_DIR ${PROJECT_SOURCE_DIR}/lib/spdlog
    CMAKE_ARGS -DCMAKE_BUILD_TYPE=${CMAKE_BUILD_TYPE}
    -DCMAKE_CXX_COMPILER=${CMAKE_CXX_COMPILER}
    -DCMAKE_C_COMPILER=${CMAKE_C_COMPILER}
    -DCMAKE_TOOLCHAIN_FILE=${CMAKE_TOOLCHAIN_FILE}
    -DCMAKE_INSTALL_PREFIX=${STAGING_DIR}
    -DSPDLOG_BUILD_SHARED=OFF
)
```

later also add:

```
add_dependencies(${PROJECT_NAME} spdlog)
```

And finally use it like this:

```
PUBLIC ${STAGING_DIR}/include/ # spdlog
```

Inside the C++/H files you can:

```
#include "spdlog/spdlog.h"
```

Using spdlog in a shared library

When building a shared library, `CMAKE_POSITION_INDEPENDENT_CODE` has to be set to `TRUE` before adding the `spdlog` subdirectory.

Full example:

```
cmake_minimum_required(VERSION 3.10)

project(cmake_test)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED TRUE)
set(CMAKE_POSITION_INDEPENDENT_CODE TRUE)

add_subdirectory("${CMAKE_SOURCE_DIR}/spdlog")

file(GLOB_RECURSE SOURCES "${CMAKE_CURRENT_SOURCE_DIR}/src/*.cpp")
add_library(test SHARED ${SOURCES})
```

```
target_include_directories(test PRIVATE "${CMAKE_CURRENT_SOURCE_DIR}/src")

target_link_libraries(test PRIVATE spdlog::spdlog)
```

Default-logger

For convenience, spdlog creates a default global logger (to stdout, colored and multithreaded).

It can be used easily by calling `spdlog::info(..), spdlog::debug(..), etc` directly.

It's instance can be replaced to any other logger (shared_ptr):

```
spdlog::set_default_logger(some_other_logger);
spdlog::info("Use the new default logger");
```

Error-handling

Spdlog will not throw exceptions while logging (since version 39cdd08).

It might throw during the construction of a logger or sink, because it is considered fatal.

If an error happens during logging, the library will print an error message to stderr. To avoid flooding the screen with error messages, the rate is limited per logger to 1 message/minute.

This behaviour can be changed by calling `spdlog::set_error_handler(new_handler_fun)` or `logger->set_error_handler(new_handler_fun)` :

Globally change the error handler:

```
spdlog::set_error_handler([](const std::string& msg) {
    std::cerr << "my err handler: " << msg << std::endl;
});
```

For a specific logger:

```
critical_logger->set_error_handler([](const std::string& msg) {
    throw std::runtime_error(msg);
});
```

The default error handler

`_default_err_handler` will use this to print error

```
fmt::print(stderr, "[*** LOG ERROR ***] [{}] [{}] {}\n", date_buf, name(), msg);
```

Home

Welcome to spdlog wiki

Visit the links on the right to see the docs about the library.

You are welcome to edit, fix, or improve them.

How-to-use-spdlog-in-DLLs

Problem

Because spdlog is header-only, building shared libraries and using it in a main program will not share the registry between them. This means that calls to functions like `spdlog::set_level(spdlog::level::level_enum::info)` will not change loggers in DLLs.

Workaround

What can be done is to register the logger in both registries.

```
/*
 * Disclaimer:
 *   This was not compiled but extracted from documentation and some code.
 */

// mylibrary.h
// In library, we skip the symbol exporting part

#include <memory>
#include <vector>
#include <spdlog/spdlog.h>
#include <spdlog/logger.h>
#include <spdlog/sinks/stdout_color_sinks.h>

namespace library
{
static const std::string logger_name = "example";

std::shared_ptr<spdlog::logger> setup_logger(std::vector<spdlog::sink_ptr> sinks)
{
    auto logger = spdlog::get(logger_name);
    if(not logger)
    {
        if(sinks.size() > 0)
        {
            logger = std::make_shared<spdlog::logger>(logger_name,
                                                      std::begin(sinks),
                                                      std::end(sinks));
            spdlog::register_logger(logger);
        }
        else
        {
            logger = spdlog::stdout_color_mt(logger_name);
        }
    }

    return logger;
}

void test(std::string message)
{
    auto logger = spdlog::get(logger_name);
    if(logger)
    {
        logger->debug("{}::{}", __FUNCTION__, message);
    }
}

}
```

```
// In the main program

#include <mylibrary.h>
#include <spdlog/logger.h>
#include <spdlog/sinks/daily_file_sink.h>
#include <spdlog/sinks/stdout_sinks.h>
```

```
int main()
{
    // We assume that we load the library here
    ...

    // Let's use the library
    std::vector<spdlog::sink_ptr> sinks;
    sinks.push_back(std::make_shared<spdlog::sinks::stdout_sink_st>());
    sinks.push_back(std::make_shared<spdlog::sinks::daily_file_sink_st>("logfile", 23, 59));

    auto logger = library::setup_logger(sinks);

    spdlog::set_level(spdlog::level::level_enum::debug); // No effect for the library.
    library::test("Hello World!"); // No logging

    spdlog::register_logger(logger);

    // Now this will also affect the library logger
    spdlog::set_level(spdlog::level::level_enum::debug);

    library::test("Hello World!"); // Hurray !

    return 0;
}
```

Integrating-into-Eclipse-IDE

This short guide can help you get started with integrating the C++ spdlog library into the Eclipse IDE. The whole configuration consists of 5 simple steps.

Note: We can use git clone. Of course :)
Note: In many cases, it is advisable to choose a parent folder named '*ThirdParty*' or '*libs*' or any other name that immediately tells us that it is a folder with third-party files. I chose as the parent folder, a folder called '*ThirdParty*'...

1. We will copy/unpack the content of the archive, which we previously downloaded from the project pages, into the newly created or already existing project. Or you can clone it from github
2. From the menu bar choose `Project > Properties > C/C+ Build > GCC C++ Compiler > Preprocessor`, and here you have to add a definition by clicking the Add button (define symbol `-D`) and add the following definition `SPDLOG_COMPILED_LIB`. Attention, keep the size of the letters!
3. Then click `Includes` (We are still here: `Project > Properties > C/C++ Build > GCC C++ Compiler`) and add the path to the folder with headers for the library spdlog. For example: `thirdParty/spdlog/include`
4. Click on `apply and close`.
5. Click `Project > Indexer > rebuild`

Happy Coding ;)

Language-bindings

Language binding

Python

On a reasonably sized log message (under 1000 bytes) spdlog will take to complete a single log transaction around 4% (async mode enabled) and 7% (sync mode) of the time required by python's standard logging logger (FileLogger).

Installation from pypi.org

`pip install spdlog`

Source

GitHub repo: spdlog-python

Features

- Async mode support
- ConsoleLogger
- FileLogger
- DailyLogger
- RotatingLogger
- SyslogLogger
- LogLevels
- Sinks

Missing:

- String formatting

Makefile-Only

Makefile

The library can also be utilized via Makefile like so:

Makefile

```
LDFLAGS += -L/path/to/lib -lspdlog
```

main.cpp

```
#define SPDLOG_COMPILED_LIB 1
#include "spdlog/spdlog.h"

// ... lots of amazing code.
```

Preventing-child-processes-from-inheriting-open-log-files

In *tweakme.h*, uncomment the following to prevent child processes from inheriting log file descriptors

```
#define SPDLOG_PREVENT_CHILD_FD
```

Preventing-undefined-thread-ids-in-child-processes

By default spdlog saves thread ids in thread local storage to gain a few micros for each call, but if your program forks, you must uncomment the `SPDLOG_DISABLE_TID_CACHING` flag in tweakme.h.

This will prevent undefined thread ids in child log messages:

```
#define SPDLOG_DISABLE_TID_CACHING
```

Setting-up-JSON-logging-with-spdlog

I haven't seen any json examples anywhere with spdlog so I thought I'd share how we did it. It's not hard to have human-readable and json-based machine-readable logs.

```
// Set up the opening brace and an array named "log"
// we're setting a global format here but as per the docs you can set this on an individual log as well
spdlog::set_pattern(set_pattern("{\n \"log\": ["");

auto mylogger = spdlog::basic_logger_mt("json_logger", "mylog.json");
mylogger->info(""); // this initializes the log file with the opening brace and the "log" array as above

// We have some extra formatting on the log level %l below to keep color coding when dumping json to the console and we use a full ISO 8601 tim
std::string jsonpattern = {"{\"time\": \"%Y-%m-%dT%H:%M:%S.%f%z\", \"name\": \"%n\", \"level\": \"%^%l%$\", \"process\": %P, \"thread\": %t, \"

spdlog::set_pattern(jsonpattern);
```

Then, log whatever you like as normal, for example:

```
mylogger->info("We have started.");
```

This will give you a log entry structured like this, although it will all be on one line:

```
{
    "time": "2021-01-10T13:44:14.567117-07:00",
    "name": "json_logger",
    "level": "info",
    "process": 6828,
    "thread": 23392,
    "message": "We have started."
}
```

You have to make sure yourself whatever you put in your log messages is valid json, you can make those as complex as you need with a complete json object if necessary. Most C++ json libraries have the ability to dump out a std::string that can be parsed as json and can then be passed as an argument to spdlog, or you can just do plain text messages like this example.

When you're finished logging, you have to close out the "log" array. We also drop the log to clean it up ourselves:

```
auto mylogger = spdlog::get("json_logger");

// All we're doing below is setting the same log format, without the "," at the end
std::string jsonlastlogpattern = { "{\"time\": \"%Y-%m-%dT%H:%M:%S.%f%z\", \"name\": \"%n\", \"level\": \"%^%l%$\", \"process\": %P, \"thread\"
spdlog::set_pattern(jsonlastlogpattern);

// below is our last log entry
mylogger->info("Finished.");

// set the last pattern to close out the "log" json array and the closing brace
spdlog::set_pattern("]\n}");

// this writes out the closed array to the file
mylogger->info("");
spdlog::drop("json_logger");
```

You end up with a log file that looks something like this (our setup and drop is done on a different thread from actual work, hence the different thread ids) :

```
{
  "log": [
    {
      "time": "2021-01-10T13:44:14.567117-07:00",
      "name": "json_logger",
      "level": "info",
      "process": 6828,
```

```
      "thread": 23392,
      "message": "We have started."
    },
    {
      "time": "2021-01-10T13:44:23.932518-07:00",
      "name": "json_logger",
      "level": "info",
      "process": 6828,
      "thread": 8048,
      "message": "We are doing something."
    },
    {
      "time": "2021-01-10T13:44:26.927726-07:00",
      "name": "json_logger",
      "level": "info",
      "process": 6828,
      "thread": 8048,
      "message": "Look a number 123.456"
    },
    {
      "time": "2021-01-10T13:44:29.631340-07:00",
      "name": "json_logger",
      "level": "info",
      "process": 6828,
      "thread": 23392,
      "message": "Finished."
    }
  ]
}
```


You have to watch what minimum log level you have enabled in your compile, if you've disabled "info" then you'll need to use a higher severity to set up the initial json and close it out at the end.

Windows-unicode-filenames

spdlog supports unicode filenames under windows. To enable it please uncomment the

```
#define SPDLOG_WCHAR_FILENAMES
```

line in the `tweakme.h` file and use the SPDLOG_FILENAME_T (or L..) macro when specifying filenames:

```
auto file_logger = spd::rotating_logger_mt("file_logger", L"logs/mylogfile", 1048576 * 5, 3);
auto file_logger2 = spd::rotating_logger_mt("file_logger2", SPDLOG_FILENAME_T("logs/mylogfile2"), 1048576 * 5, 3);
```